TrendPilot

A Local Agentic AI Assistant Using Ollama and Gemma 3:4B

AIRI Team PITB - AI Internship Task 2

End-to-End Agentic AI Project: Final Report

Prepared by: Musferah Akram

Submitted to: Omar Farooq, Associate AI/ML Engineer

Contents

Part 1: Final Project Report

- 1. Problem Understanding and Project Idea
- 2. Ollama Setup and Local Model Usage
- 3. Agent Workflow Design
- 4. Tool Implementation
- 5. Memory and File Saving
- 6. Demo Interface
- 7. Testing (10 test cases)
- 8. Error Analysis and Improvements
- 9. Conclusion
- 10. Project Structure and Deliverables Checklist
- Appendix A: Full Test Outputs

Part 2: LinkedIn Portfolio Report

Part 1: Final Project Report

This part documents the project as required by the assignment: the setup, workflow, tools, demo, test results with outputs and screenshots, and the failure analysis.

1. Problem Understanding and Project Idea

Objective. The task is to build a local agentic AI assistant using a small language model through Ollama. An agent differs from a chatbot because it breaks a goal into steps, calls tools, stores information, reviews its own output and saves the result.

Project chosen. I built TrendPilot (Option A in the assignment): a local agent that takes a topic from the user and produces a content plan for platforms such as LinkedIn, Instagram, X and YouTube Shorts. Many students finish a technical project but struggle to turn it into a clear post. TrendPilot creates a first draft of that content, and everything runs on the local machine.

Inputs and outputs. The user gives a topic, platform, tone and output type (post, reel script, or both), and optionally extra details. The agent produces a content idea, target audience, hook, caption, hashtags, reel script, title, thumbnail text and review comments, and saves the result to a file.

Minimum requirements check

Requirement	Required by the assignment	This project
Local LLM	Ollama model	Ollama
Recommended model	Gemma 3:4B	gemma3:4b
Agent tools	At least 3, File Saver mandatory	12 tools, including File Saver
Memory	Required	Memory Reader and Memory tools
Demo interface	CLI, Streamlit or FastAPI	CLI (python demo.py)
Test cases	At least 10	10
Saved outputs	At least 5	10 in saved_results (plus 7 posts and 3 scripts in their own folders)
Final report	Required	This document
GitHub or Drive submission	Required	GitHub repository/Drive

2. Ollama Setup and Local Model Usage

The project uses Ollama with the recommended model gemma3:4b. It ran on my laptop, so no substitute model was needed. The agent's Python code sends its prompts to this local model, and no online AI service is used.

To confirm the model runs, I started it with ollama run gemma3:4b and used the assignment's test prompt, "Give me 5 viral LinkedIn post ideas about AI internships." The model answered locally (Figure 1).

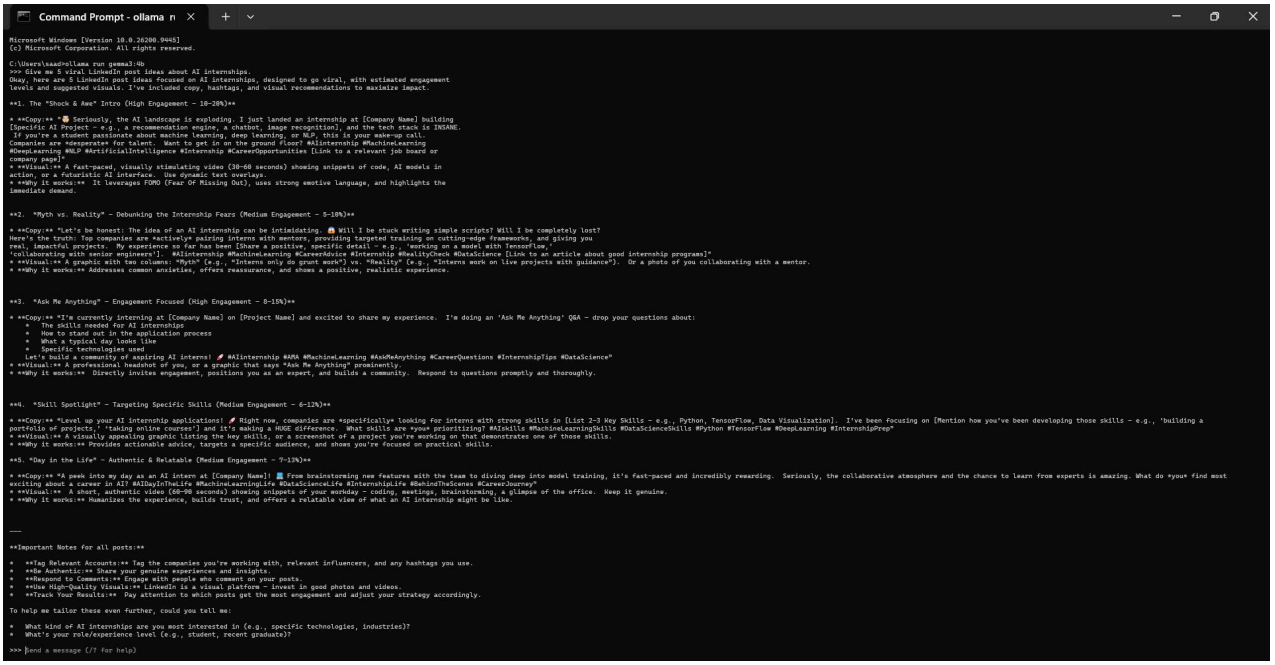
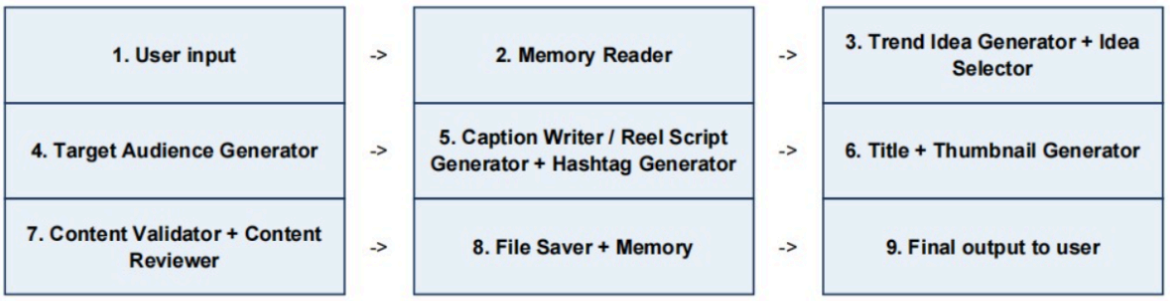


Figure 1: Ollama running gemma3:4b and answering the test prompt from the assignment.

3. Agent Workflow Design

The agent follows a fixed plan of steps. The tool logs show the same order in every test, and each tool call is printed to the terminal as [Tool Called]: Tool Name, so the workflow can be followed live. Read the diagram left to right, top to bottom.



For a script-only request the Caption Writer is skipped, and for a post-only request the Reel Script Generator is skipped. The Content Validator checks the result automatically against the platform rules, and the Content Reviewer then reviews it and suggests improvements before the File Saver stores it.

Prompt rules

All prompts and platform rules are kept in one file (prompts.py) so every tool follows the same guidance. It holds:

- A system prompt with hard rules: only state facts the user gave, never invent statistics or results, no greetings such as "Okay, here's", and no placeholder text.
- Platform rules with a length limit, hashtag limit and writing guide for LinkedIn, Instagram, X, YouTube Shorts, TikTok and Facebook (for example, X is limited to 220 characters and 2 hashtags).
- A humor guide, used for funny and meme-style tones.

4. Tool Implementation

TrendPilot has 12 tools. The assignment requires at least three, and all eight tools it lists are implemented. Idea Selector, Target Audience Generator and Content Validator are extra tools I added.

Tool	What it does
Memory Reader	Reads previous topics and outputs before a new request starts.
Trend Idea Generator	Asks the model for several content angles for the topic and platform.
Idea Selector	Chooses the best angle to write about.
Target Audience Generator	Describes who the post or video is aimed at.
Caption Writer	Writes the caption using the platform guide and the chosen tone.
Hashtag Generator	Returns hashtags, limited to the platform maximum.
Reel Script Generator	Writes a short video script with hook, main explanation, visual suggestion and closing line.
Title + Thumbnail Generator	Suggests a title and thumbnail text.
Content Validator	Automatic checks against platform rules such as caption length and hashtag count. It reported 0 warnings in all 10 tests.
Content Reviewer	The model reviews hook, clarity, relevance, tone, length and unsupported claims, and suggests improvements.
File Saver (mandatory)	Saves the final result as a Markdown file in the outputs folder.
Memory	Records the request so it can be read back in later runs.

Coverage of the assignment's tool list

Tool in the assignment (Section 6)	Implemented as
Tool 1: Trend Idea Generator	Trend Idea Generator
Tool 2: Hashtag Generator	Hashtag Generator
Tool 3: Caption Writer	Caption Writer
Tool 4: Reel Script Generator	Reel Script Generator
Tool 5: Title and Thumbnail Text Generator	Title + Thumbnail Generator
Tool 6: File Saver (mandatory)	File Saver
Tool 7: Content Reviewer	Content Reviewer
Tool 8: Simple Memory Tool	Memory and Memory Reader

5. Memory and File Saving

Memory. At the start of a run the Memory Reader loads earlier topics and outputs, and at the end the Memory tool records the new request. The project has a dedicated memory folder for this.

File saving. The File Saver writes each result as a Markdown file with sections such as Hook, Target Audience, Caption and Hashtags (see Figure 3). Every file name ends with a timestamp, for example YOLOv8_helmet_detection_20260929_014557.md, so a new run never overwrites an old one.

Output folders. All 10 test runs saved a file in outputs/saved_results. Post outputs are also saved in outputs/generated_posts (7 files) and script outputs in outputs/generated_scripts (3 files). Earlier Round 1 results are kept in outputs/saved_results/archive_v1.

6. Demo Interface

The demo is a command-line app. From the project folder, run `python demo.py`. The terminal asks for the topic, platform, tone and output type, prints each [Tool Called] line, then prints the final result and the saved file.

Example run (Figure 2). Topic: YOLOv8 helmet detection project. Platform: LinkedIn. Tone: Professional. Output type: Post + Reel Script. Details: trained YOLOv8 on a vehicles dataset in Google Colab and tested on unseen images.

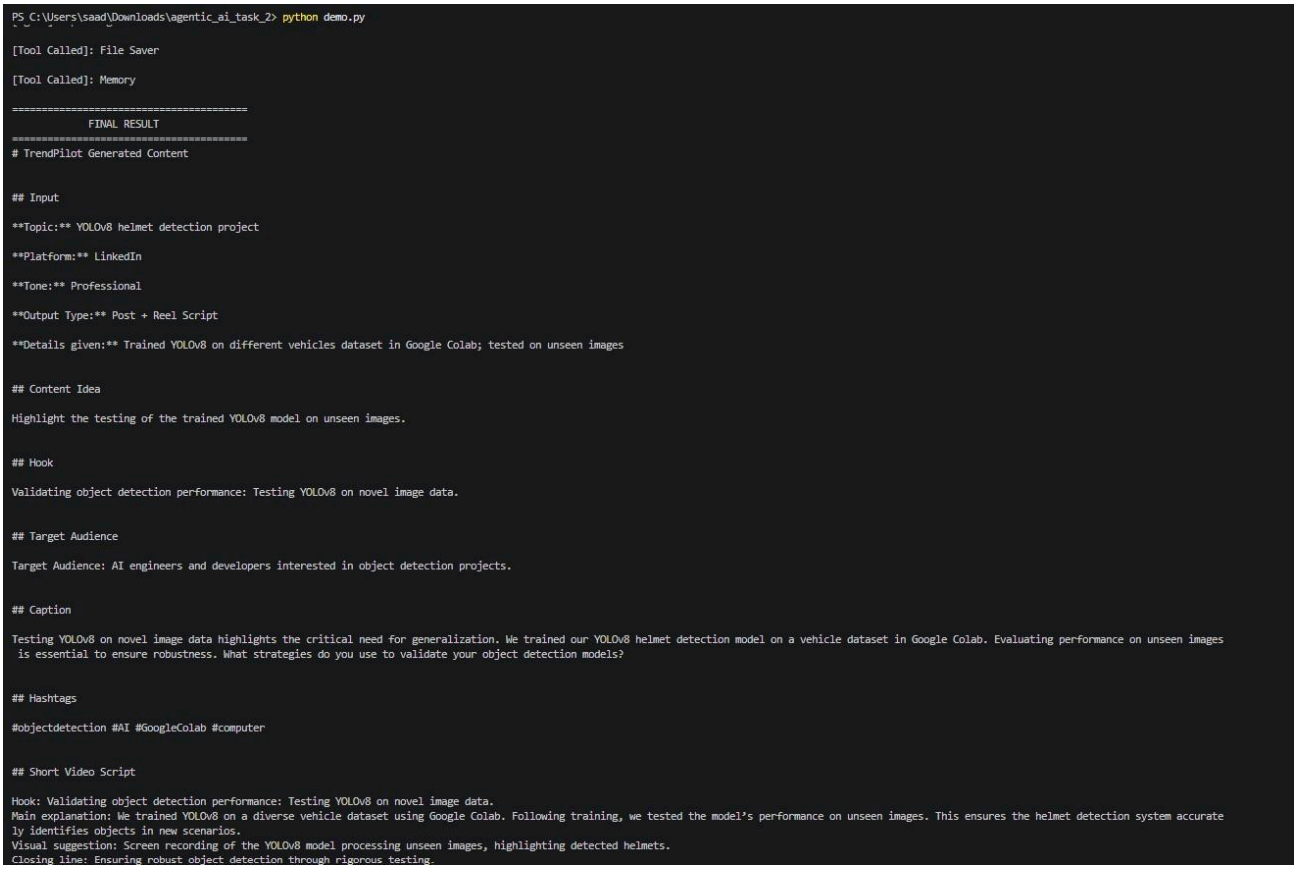


Figure 2: python demo.py showing the [Tool Called] lines and the FINAL RESULT for the YOLOv8 helmet detection example.

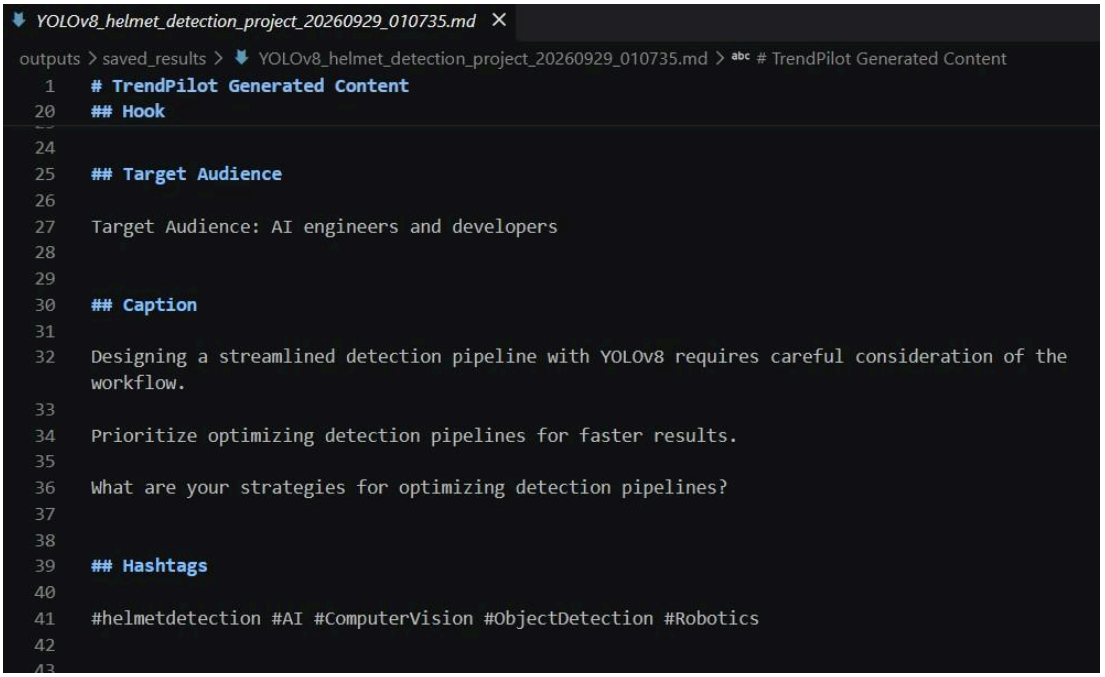


Figure 3: The saved Markdown file (YOLOv8_helmet_detection_project_20260929_010735.md) in outputs/saved_results.

7. Testing

I tested the agent on 10 different inputs using tests/run_tests.py, which runs every case and saves the results to tests/test_results.md. All 10 tests ran to completion (PASS) with 0 automatic check warnings, and the average run time was about 142 seconds per test. For each test I recorded the input, tools used, final output, saved file path and whether the result was useful (Appendix A). I judged 5 outputs useful and 5 not useful.

#	Topic	Platform / Tone	Output	Status	Useful?
1	YOLOv8 helmet detection	LinkedIn / Professional	Post	PASS (145s)	No
2	AI internship experience	LinkedIn / Motivational	Post	PASS (128s)	No
3	Python learning journey	Instagram / Funny	Reel Script	PASS (151s)	Yes
4	Machine learning project	X / Short and catchy	Post	PASS (129s)	No
5	Data annotation struggles	LinkedIn / Humorous	Post	PASS (160s)	No
6	Model failed in production	Instagram / Meme style	Reel Script	PASS (146s)	No
7	Final year project	LinkedIn / Professional	Post	PASS (138s)	Yes
8	Computer vision project	YouTube Shorts / Exciting	Reel Script	PASS (146s)	Yes
9	Debugging FastAPI	X / Funny	Post	PASS (137s)	Yes
10	Open-source contribution	LinkedIn / Inspirational	Post	PASS (139s)	Yes

Judgment for each test

#	Useful?	Reason
1	No	Generic caption with no detail about the helmet project and a weak hook.
2	No	Generic, and it invents an internship lesson (teamwork) that was never given.
3	Yes	Funny opening and a clear call to action, but it drifts into data cleaning instead of Python.
4	No	Reads like generic advice, not a catchy X post.

#	Useful?	Reason
5	No	Meant to be humorous but has no joke, and the lines repeat each other.
6	No	Not meme style, just a generic deployment tip.
7	Yes	Professional and clean, but it talks about documentation instead of the project itself.
8	Yes	Has all four script parts (hook, explanation, visual, closing line), though the wording is bland.
9	Yes	Short, one real joke, relevant hashtags, and fits X.
10	Yes	On topic, inspirational, and ends with a good question.

Strong outputs: Test 9 (a short X joke with relevant hashtags), Test 10 (on topic and inspirational), Test 3 (funny opening and clear call to action) and Test 8 (all four script parts present).

Weak outputs: Tests 5 and 6 (humor and meme style with no real joke), and Tests 1, 2 and 4 (generic captions).

8. Error Analysis and Improvements

I reviewed the 5 weak outputs from the test run. The table shows what went wrong, the likely reason, and what the prompt rules did about it.

Test Case	Problem Found	Possible Reason	Fix Applied / Prompt Rule
Test 1: YOLOv8 helmet detection	Generic caption with no project detail.	Only the topic was given, and the rule says to stay general when few facts are provided.	Rule: only state facts the user gave. It stops false claims but leaves the caption generic.
Test 2: AI internship experience	Invented a teamwork lesson the user never gave.	The small model ignored the rule against unstated claims.	Same rule in prompts.py. It only partly worked.
Test 4: Machine learning project (X)	Round 1 invented customer-data claims and attached a reel script. Round 2 is short but reads like advice.	Vague topic, and the model wrote instructions instead of a hook.	Added the no-invented-facts rule and X limits (220 characters, 2 hashtags). Fixed the invented claims, not the style.
Test 5: Data annotation struggles	Humorous tone requested, but no joke and the lines repeat.	The 4B model does not follow the humor guide well.	Added a humor guide with an example joke. It did not fix the problem.
Test 6: Model failed in production	Round 1 read "production" as film or factory work. Round 2 is not meme style.	The model misread "production", and meme format is hard for a small model.	Added a rule that production means deployed software, plus the humor guide. Fixed the meaning, not the meme style.

Round 1(First Version) compared with Round 2(Final Version)

I compared the archived Round 1 outputs for Tests 4 and 6 with their Round 2 versions:

Aspect	Round 1 (archive_v1)	Round 2 (final tests)
Meaning of "production" (Test 6)	Robot arm and film-set style content; hashtags such as #FilmProductionProblems and #OnSetIncident.	Treated as a deployed model; hashtags such as #SoftwareEngineering and #ModelDebugging.
X post (Test 4)	Reel script attached, 10 hashtags, and an invented claim that customer behavior data was being analyzed with promising results.	One short line, 2 hashtags, no reel script, no invented results.
Model chatter	Outputs contained lines such as "Okay, here's..." and "Do you want me to tweak this?".	None of the 10 test outputs contain such lines.

What improved and what is still weak

- **Improved:** X posts no longer come with an unrelated reel script; hashtags stay within the platform limit; no invented statistics, metrics or tool names appeared in the 10 outputs; the outputs are clean text without model chatter.
- **Still weak:** many captions are generic (Tests 1, 2, 4); humor and meme style are weak (Tests 5, 6); a few captions still claim small personal experiences the user never gave (Tests 2 and 7); some hashtag sets are generic or very short (Tests 3 and 8).

Suggested improvements

- **Better input:** add an optional "project details" field so the agent has real facts to write from instead of staying generic.
- **Better humor:** add several example jokes and meme formats to the prompt, or try a larger local model.
- **Validation check:** add a check that flags personal claims (such as an internship lesson) that the user never provided.
- **User feedback loop:** let the user ask for a rewrite of a weak caption.

9. Conclusion

TrendPilot meets the assignment requirements: it uses a local model through Ollama, runs a multi-step workflow with 12 tools, keeps memory, saves every result to a file, and has a working CLI demo. All 10 test cases ran successfully. The agent worked best on short, structured technical topics (5 of 10 outputs were useful) and was weakest on humor, meme style and topics where the user gives few details. The main limits are the small 4B model and the lack of project details in the input, and the improvements in Section 8 address both.

10. Project Structure and Deliverables Checklist

Deliverable (Section 20)	Where it is
GitHub repository or Drive folder link	GitHub repository, link sent to Sir Omar Farooq.
Complete source code	app/ (main.py, agent.py, tools.py, memory.py, prompts.py) and demo.py.
requirements.txt	Project root.
README file	Project root (README.md).
Working demo (CLI)	python demo.py (Section 6).
Screenshot: Ollama model running	screenshots/ollama_running.png (Figure 1).
Screenshots of agent outputs	screenshots/demo_output.png and saved_file_output.png (Figures 2 and 3).
At least 5 saved outputs	outputs/saved_results (10 test files), generated_posts (7), generated_scripts (3).
Test results for 10 inputs	Section 7, Appendix A and tests/test_results.md.
Error-analysis table	Section 8 and tests/test_cases.md.
Final report PDF	This document (report/final_report.pdf).
LinkedIn-ready project report section	Part 2 of this document.

Project structure

```
agentic_ai_task_2/
  app/            __init__.py    agent.py  main.py  memory.py  prompts.py  tools.py
  memory/         history.json
  outputs/        generated_posts/  generated_scripts/  saved_results/ (archive_v1/)
  report/         final_report.pdf
  screenshots/    ollama_running.png  demo_output.png  saved_file_output.png
  tests/          run_tests.py  test_cases.md  test_results.md
  .gitignore
  README.md
  requirements.txt
  demo.py
```

Appendix A: Full Test Outputs

Each entry shows the input, the tools used, the status, the saved file path, my judgment, and the final output.

Test 1: YOLOv8 helmet detection

Input: topic = YOLOv8 helmet detection; platform = LinkedIn; tone = Professional; output = Post

Status: PASS (145s)

Tools used: Memory Reader, Trend Idea Generator, Idea Selector, Target Audience Generator, Caption Writer, Hashtag Generator, Title + Thumbnail Generator, Content Validator, Content Reviewer, File Saver, Memory

Saved file: outputs/saved_results/YOLOv8_helmet_detection_20260929_014557.md

Useful? No. Generic caption with no detail about the helmet project and a weak hook.

Caption:

YOLOv8's speed and accuracy are key for real-time object detection.

Utilizing this approach provides a balance between identifying individuals wearing helmets and maintaining processing efficiency.

What are your thoughts on applying this method to similar applications?

Hashtags: #helmetdetection #AI #ComputerVision #ObjectDetection #Robotics

Test 2: AI internship experience

Input: topic = AI internship experience; platform = LinkedIn; tone = Motivational; output = Post

Status: PASS (128s)

Tools used: Memory Reader, Trend Idea Generator, Idea Selector, Target Audience Generator, Caption Writer, Hashtag Generator, Title + Thumbnail Generator, Content Validator, Content Reviewer, File Saver, Memory

Saved file: outputs/saved_results/AI_internship_experience_20260929_014805.md

Useful? No. Generic, and it invents an internship lesson (teamwork) that was never given.

Caption:

Building AI solutions reshaped my approach to problem-solving.

The internship highlighted the value of teamwork in development.

What's one thing you've learned recently that shifted your perspective?

Hashtags: #ArtificialIntelligence #TechInternships #SoftwareDevelopment #MachineLearning

Test 3: Python learning journey

Input: topic = Python learning journey; platform = Instagram; tone = Funny; output = Reel Script

Status: PASS (151s)

Tools used: Memory Reader, Trend Idea Generator, Idea Selector, Target Audience Generator, Hashtag Generator, Reel Script Generator, Title + Thumbnail Generator, Content Validator, Content Reviewer, File Saver, Memory

Saved file: outputs/saved_results/Python_learning_journey_20260929_015036.md

Useful? Yes. Funny opening and a clear call to action, but it drifts into data cleaning instead of Python.

Hashtags: #learning #programming #development #coding #software #Python

Script:

Data cleaning is my cardio – it's exhausting! Spending hours fixing messy data is like endless debugging. I'm learning Python, and it's way tougher than I thought. Picture this: a chaotic spreadsheet… Let's hear your toughest data cleaning stories!

Test 4: Machine learning project

Input: topic = Machine learning project; platform = X; tone = Short and catchy; output = Post

Status: PASS (129s)

Tools used: Memory Reader, Trend Idea Generator, Idea Selector, Target Audience Generator, Caption Writer, Hashtag Generator, Title + Thumbnail Generator, Content Validator, Content Reviewer, File Saver, Memory

Saved file: outputs/saved_results/Machine_learning_project_20260929_015245.md

Useful? No. Reads like generic advice, not a catchy X post.

Caption:

Build a machine learning project. Start with your data, train your model, and deploy it.

Hashtags: #MachineLearning #AIProject

Test 5: Data annotation struggles

Input: topic = Data annotation struggles; platform = LinkedIn; tone = Humorous; output = Post

Status: PASS (160s)

Tools used: Memory Reader, Trend Idea Generator, Idea Selector, Target Audience Generator, Caption Writer, Hashtag Generator, Title + Thumbnail Generator, Content Validator, Content Reviewer, File Saver, Memory

Saved file: outputs/saved_results/Data_annotation_struggles_20260929_015525.md

Useful? No. Meant to be humorous but has no joke, and the lines repeat each other.

Caption:

Is meticulous data labeling exhausting? It is.

Spending hours precisely labeling data demands intense focus and attention to detail.

Data annotation requires a concentrated effort.

What's the most challenging part of your data labeling work?

Hashtags: #AIchallenges #dataproblems #annotationstruggles #datascience

Test 6: Model failed in production

Input: topic = Model failed in production; platform = Instagram; tone = Meme style; output = Reel Script

Status: PASS (146s)

Tools used: Memory Reader, Trend Idea Generator, Idea Selector, Target Audience Generator, Hashtag Generator, Reel Script Generator, Title + Thumbnail Generator, Content Validator, Content Reviewer, File Saver, Memory

Saved file: outputs/saved_results/Model_failed_in_production_20260929_015751.md

Useful? No. Not meme style, just a generic deployment tip.

Hashtags: #ProductionIssues #SoftwareEngineering #AIIncident #DeploymentFailure #TechSupport #SystemError #ModelDebugging

Script:

Deploying a model is like sailing… only with more debugging.

You spend weeks perfecting it, and then it fails in production. Unexpected issues are common.

Ensure your model performs as expected after deployment.

Level up your deployment process.

Test 7: Final year project

Input: topic = Final year project; platform = LinkedIn; tone = Professional; output = Post

Status: PASS (138s)

Tools used: Memory Reader, Trend Idea Generator, Idea Selector, Target Audience Generator, Caption Writer, Hashtag Generator, Title + Thumbnail Generator, Content Validator, Content Reviewer, File Saver, Memory

Saved file: outputs/saved_results/Final_year_project_20260929_020009.md

Useful? Yes. Professional and clean, but it talks about documentation instead of the project itself.

Caption:

Architecting complex systems demands meticulous documentation.

My final year project underscored the value of detailed records during development.

Focus on clear, traceable documentation – it's a key to successful project completion.

What's your approach to technical documentation?

Hashtags: #FinalYearProject #AIProject #SoftwareDevelopment #TechProject #EngineeringProject

Test 8: Computer vision project

Input: topic = Computer vision project; platform = YouTube Shorts; tone = Exciting; output = Reel Script

Status: PASS (146s)

Tools used: Memory Reader, Trend Idea Generator, Idea Selector, Target Audience Generator, Hashtag Generator, Reel Script Generator, Title + Thumbnail Generator, Content Validator, Content Reviewer, File Saver, Memory

Saved file: outputs/saved_results/Computer_vision_project_20260929_020235.md

Useful? Yes. Has all four script parts (hook, explanation, visual, closing line), though the wording is bland.

Hashtags: #AI #machinelearning #vision

Script:

Hook: See object recognition in action!

Main explanation: Start with images. Train a model to identify objects. The system recognizes objects in new images.

Visual suggestion: Fast montage of objects, code snippets, and a basic object detection visualization.

Closing line: Discover computer vision.

Test 9: Debugging FastAPI

Input: topic = Debugging FastAPI; platform = X; tone = Funny; output = Post

Status: PASS (137s)

Tools used: Memory Reader, Trend Idea Generator, Idea Selector, Target Audience Generator, Caption Writer, Hashtag Generator, Title + Thumbnail Generator, Content Validator, Content Reviewer, File Saver, Memory

Saved file: outputs/saved_results/Debugging_FastAPI_20260929_020452.md

Useful? Yes. Short, one real joke, relevant hashtags, and fits X.

Caption:

FastAPI debugging: Tracebacks and I are locked in a serious debate.

Hashtags: #FastAPI #Debugging

Test 10: Open-source contribution

Input: topic = Open-source contribution; platform = LinkedIn; tone = Inspirational; output = Post

Status: PASS (139s)

Tools used: Memory Reader, Trend Idea Generator, Idea Selector, Target Audience Generator, Caption Writer, Hashtag Generator, Title + Thumbnail Generator, Content Validator, Content Reviewer, File Saver, Memory

Saved file: outputs/saved_results/Open_source_contribution_20260929_020711.md

Useful? Yes. On topic, inspirational, and ends with a good question.

Caption:

It’s intimidating starting with open-source contributions.

Early projects demand a steep learning curve.

Collaboration offers a powerful way to grow your skills.

Focus on consistent learning and reward yourself for progress.

What’s the biggest lesson you took from your first contribution?

Hashtags: #softwaredevelopment #contribution #techcommunity #AIdevelopment

Part 2: LinkedIn Portfolio Report

This part is a short portfolio write-up that will/can be used as a LinkedIn post, a GitHub README summary, or a portfolio page.

Project Title

TrendPilot: A Local Agentic AI Assistant Using Ollama

1. Project Overview

I built TrendPilot, a local agentic AI assistant that runs on my laptop using Ollama and Gemma 3:4B. It solves a common problem: finishing a technical project but not knowing how to present it online. The user enters a topic, platform, tone and output type, and the agent uses tools such as an idea generator, caption writer, hashtag generator, reel script generator, content reviewer and file saver. The final output is a content plan saved as a Markdown file. The goal was to understand how agentic workflows work beyond simple chatbot responses.

2. Tools and Technologies Used

- Python
- Ollama
- Gemma 3:4B
- Command-line (CLI) demo
- Memory tool for previous topics and outputs
- Markdown file saving
- GitHub

Tools used: Python, Ollama, Gemma 3:4B, a CLI demo, a memory tool, Markdown file saving, and GitHub. LangChain, LangGraph, Streamlit, FastAPI and SQLite were not used.

3. Agent Workflow

- User gives topic, platform, tone and output type
- Agent follows a fixed plan of steps
- Agent selects and calls tools, and each call is printed to the terminal
- Tools generate the idea, audience, caption or script, hashtags, and title
- Validator and reviewer check the output and suggest improvements
- File saver stores the result as a Markdown file
- Final output is returned to the user

The agent first reads its memory, then generates content angles and picks the best one. It writes a caption or reel script, creates hashtags, suggests a title and thumbnail text, checks and reviews the quality, and saves the final result as a Markdown file.

4. Tools Implemented

- **Memory Reader:** loads earlier topics and outputs.
- **Trend Idea Generator:** proposes content angles for the topic.
- **Idea Selector:** picks the best angle.
- **Target Audience Generator:** says who the content is for.
- **Caption Writer:** writes the caption for the chosen platform and tone.
- **Hashtag Generator:** creates hashtags within the platform limit.
- **Reel Script Generator:** writes a short script with hook, explanation, visual suggestion and closing line.
- **Title + Thumbnail Generator:** suggests a title and thumbnail text.
- **Content Validator:** checks length and hashtag limits automatically.
- **Content Reviewer:** reviews hook, clarity, tone and unsupported claims.
- **File Saver:** saves the final result as a Markdown file.
- **Memory:** records the request for later runs.

5. Demo and Outputs

How the demo runs: I run python demo.py in the terminal. It asks for the topic, platform, tone and output type, shows each tool as it is called, and prints the final result.

Example input: Topic: YOLOv8 helmet detection project; Platform: LinkedIn; Tone: Professional; Output: Post + Reel Script.

Example output (hook): "Validating object detection performance: Testing YOLOv8 on novel image data."

Screenshot references: screenshots/ollama_running.png, screenshots/demo_output.png and screenshots/saved_file_output.png (Figures 1 to 3 in Part 1).

Saved output files: outputs/saved_results (10 test files), outputs/generated_posts and outputs/generated_scripts.

6. Testing

I tested the agent on 10 topics, including a YOLOv8 helmet detection post, an AI internship post, a Python learning reel, an X post about a machine learning project, a data annotation joke, a meme-style reel about a model failing in production, a computer vision short, a FastAPI debugging joke, and an open-source contribution post. The prompts covered LinkedIn, Instagram, X and YouTube Shorts, with tones from professional to funny and meme style. **Strong outputs:** the short FastAPI joke for X, the open-source post, the Python reel and the computer vision script. **Weak outputs:** the humor and meme tests (data annotation, model failed in production) and some generic captions.

7. Error Analysis

What failed: some captions were too generic, some invented small personal claims, and the humor and meme tests had no real joke. **Why:** the input often had only a topic, and a small 4B model does not follow humor instructions well. **How I improved it:** compared with my first round, I added prompt rules that stop invented facts and numbers, treat "production" as deployed software, and limit X posts to 220 characters and 2 hashtags. This removed the invented claims and the wrong film meaning, but did not fix humor or generic wording.

8. What I Learned

- Learned how to run a local LLM with Ollama and call it from Python.
- Learned the difference between a chatbot and an AI agent.
- Learned how to connect tools to an LLM and log each tool call.
- Learned how to write prompt rules for structured output, and that a small model follows some rules better than others.
- Learned how to save agent results and keep a simple memory.
- Learned how to test on different inputs, judge the results honestly and compare them with an earlier version.
- Learned that limiting invented claims can make captions more generic, so the input needs real details.

9. Future Improvements

- Add a project-details input so captions are specific.
- Improve humor and meme prompts with more examples, or try a larger local model.
- Add a check for unsupported personal claims.
- Add a user feedback loop for rewrites.
- Add a simple web interface (Streamlit) and better memory (SQLite).
- Add web search to find current trends.

10. LinkedIn Post Format

Today, I completed AIRI Team PITB - AI Internship Task 2, where I built an end-to-end Agentic AI assistant using a local LLM through Ollama.

For this task, I developed TrendPilot, an AI agent that can take a user topic and generate structured outputs such as content ideas, LinkedIn posts, hashtags, reel scripts, and saved Markdown files.

This project helped me understand how agentic AI systems work beyond simple chatbot responses. I learned how to design an agent workflow, connect tools, add memory, save outputs, test different prompts, and analyze weak responses.

I tested it on 10 topics. It handled structured technical topics reasonably well, but humor and meme style were still weak with a 4B model.

Tools used: Python, Ollama, Gemma 3:4B, CLI, Markdown, and GitHub.

Key learning: Agentic AI is not only about generating text. It is about planning, using tools, storing useful information, improving outputs, and completing tasks in a structured way.

#AgenticAI #Ollama #LocalLLM #Gemma #Python #ArtificialIntelligence #AIRI #PITB #AIInternship #MachineLearning #GenerativeAI